



PUC Minas

Introdução a P00

Renan Santos Mendes



PUC Minas

Programação Orientada a Objetos (POO)

- **Programação Orientada a Objetos** é um **paradigma de programação** baseado no conceito de **objetos**, que representam **entidades do mundo real**.
- Os objetos possuem **propriedades** (atributos) e **comportamentos** (métodos) específicos.
- O programa é organizado em **classes**, que são **modelos para a criação de objetos**, e **objetos**, que são **instâncias de uma classe**.

- Os objetos interagem entre si por meio de mensagens, que são enviadas de um objeto para outro.
- Cada objeto possui seu próprio estado e comportamento, e pode se comunicar com outros objetos para realizar ações mais complexas.

POO

- A POO aumenta a **eficiência, modularidade, flexibilidade e reutilização de código** em programas mais complexos.
- Conceitos como **herança, encapsulamento, polimorfismo e abstração** permitem criar hierarquias de classes, esconder informações internas de uma classe, criar objetos com comportamentos diferentes a partir de uma mesma classe e simplificar o código ao definir interfaces mais abstratas.

POO

- A POO é utilizada em diversas linguagens de programação, como Python, Java, C++, Ruby, entre outras.
- É especialmente útil em projetos de grande escala, onde a organização e reutilização do código são essenciais.



PUC Minas

Conceitos Fundamentais

- **Classes e objetos:** **classes** são estruturas que definem as características e comportamentos dos objetos. Os **objetos**, por sua vez, são instâncias de uma classe, ou seja, são criados a partir de uma classe.
- **Atributos e métodos:** **atributos** são as características dos objetos, como nome, idade, cor, entre outros. Já os **métodos** são as ações que os objetos podem realizar, como andar, falar, calcular, entre outros.

- **Herança:** é o processo de **criar uma nova classe a partir de uma classe existente**, mantendo suas características e comportamentos. A nova classe pode adicionar novos atributos e métodos ou sobrescrever os existentes.
- **Encapsulamento:** é o princípio de **esconder a implementação** de um objeto de outros objetos. É possível tornar certos atributos e **métodos privados**, para que só possam ser acessados e modificados pela própria classe.

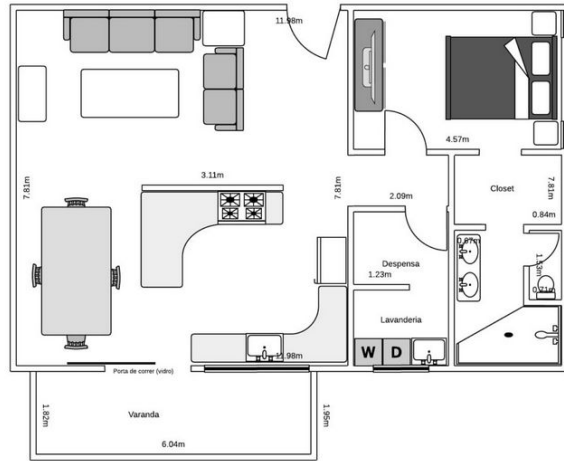


PUC Minas

Classes X Objetos

POO

Classe



Planta → Projeto

Criação de
Uma Instância



Objeto



Casa → Instância



PUC Minas

Como é feito em Python?

PОО - Implementação

```
class Carro(object):  
    def __init__(self, marca, modelo, ano):  
        self.marca = marca  
        self.modelo = modelo  
        self.ano = ano  
  
    def acelerar(self):  
        print(f'{self.marca} {self.modelo} está acelerando...')
```



PОО - Implementação

Criação da Classe

```
class Carro(object):  
    def __init__(self, marca, modelo, ano):  
        self.marca = marca  
        self.modelo = modelo  
        self.ano = ano  
  
    def acelerar(self):  
        print(f'{self.marca} {self.modelo} está acelerando...')
```



PОО - Implementação

Criação do
Construtor

```
class Carro(object):  
    def __init__(self, marca, modelo, ano):  
        self.marca = marca  
        self.modelo = modelo  
        self.ano = ano  
  
    def acelerar(self):  
        print(f'{self.marca} {self.modelo} está acelerando...')
```



P OO - Implementação

Uso da
Palavra "self"

```
class Carro(object):  
    def __init__(self, marca, modelo, ano):  
        self.marca = marca  
        self.modelo = modelo  
        self.ano = ano  
  
    def acelerar(self):  
        print(f'{self.marca} {self.modelo} está acelerando...')
```



P OO - Implementação

Criação dos
Atributos

```
class Carro(object):  
    def __init__(self, marca, modelo, ano):  
        self.marca = marca  
        self.modelo = modelo  
        self.ano = ano  
  
    def acelerar(self):  
        print(f'{self.marca} {self.modelo} está acelerando...')
```



P OO - Implementação

```
class Carro(object):  
    def __init__(self, marca, modelo, ano):  
        self.marca = marca  
        self.modelo = modelo  
        self.ano = ano  
  
    def acelerar(self):  
        print(f'{self.marca} {self.modelo} está acelerando...')
```

Criação dos
Métodos



P OO - Implementação

```
class Carro(object):  
    def __init__(self, marca, modelo, ano):  
        self.marca = marca  
        self.modelo = modelo  
        self.ano = ano  
  
    def acelerar(self):  
        print(f'{self.marca} {self.modelo} está acelerando...')
```

Uso dos Atributos
dentro dos métodos





PUC Minas